

Progetto Winsome per RCL 21/22

Classi:

Thread :

- **ServerMain:**

Il main del server implementa il ciclo per accettare nuove connessioni sul socket TCP e il conseguente avvio del thread per gestire le richieste del client.

Inoltre esporta gli oggetti remoti per la registrazione e per le callback inerenti agli eventi di follow e unfollow.

Avvia inoltre il thread per il backup, quello per il calcolo delle ricompense e quello per la chiusura del server.

- **ClientMain:**

Il main del client implementa un ciclo che permette all'utente di inserire un comando che poi verrà mandato al server, oltre a implementare tutte le funzioni inerenti a questi comandi che vanno svolte a lato client, ossia mostrare la lista dei follower dell'utente se questo ha precedentemente eseguito il login, registrarsi e deregistrarsi dalle callback e dal gruppo multi cast rispettivamente quando avviene la conferma di login o di logout.

Inoltre sempre al momento del login/logout avvia/interrompe il thread per la ricezione delle notifiche di ricalcolo delle ricompense.

- **RequestManager:**

Gestisce le richieste di un dato client ricevendole e inviando le relative risposte tramite un socket TCP. Agisce sulle mappe dei post e degli utenti, essendo previsto che ci siano molteplici utenti che interagiscono con Winsome contemporaneamente va garantita la thread safeness su queste strutture, ciò è fatto implementandole con delle concurrentHashMap. Discorso analogo per il "lastPostID" che per questo motivo è implementato con un Atomic Integer.

- **BackupManager:**

Esegue un backup periodico della mappa dei post, degli utenti, del tempo in ms dell'ultimo calcolo delle renard e dell'ultimo ID assegnato a un post.

- **ServerCloser:**

Esegue la chiusura del server nel momento in cui riceve il comando exit. Termina i vari thread, forza l'uscita dal loop per accettare nuove connessioni ed esegue un ultimo salvataggio.

- **WalletUpdateSender:**

Calcola periodicamente le ricompense relative ai vari post da consegnare a ciascun utente per poi aggiornare i loro wallet e mandare una notifica su un gruppo multi cast via UDP per notificare i registrati che i wallet sono stati aggiornati. Agisce su una copia dei post effettuata dopo aver impostato il tempo dell'ultimo calcolo delle ricompense, su ogni commento e voto verrà fatto un controllo che sia stato inserito nell'intervallo di tempo compreso tra il momento del salvataggio precedente e quello attuale, in questo modo commenti e voti aggiunti tra quando si assegna il nuovo valore al momento dell'ultimo ricalcolo e il momento in cui si effettua la copia dei post non verranno conteggiati nel calcolo corrente ma nel prossimo, affinché non vengano conteggiati due volte.

- **WalletUpdateReceiver:**

Thread lato client che attende l'arrivo di notifiche di avvenuto ricalcolo delle ricompense.

Entità :

- **SocialNetwork:**

Rappresenta il social network Winsome, mantiene una mappa di utenti e una di post oltre a altre informazioni di utilità.

- **User:**

Rappresenta un utente del social network e ne memorizza le informazioni, oltre a implementare la maggioranza dai metodi che il client può richiedere al sistema.

Nello specifico memorizza quattro set di stringhe, uno per i tags, uno per gli utenti seguiti, uno per quelli che lo seguono e uno per gli ID dei post, che li identificano nella mappa del social network, nel proprio blog.

- **Post:**

Mantiene una mappa Stringa -> Lista di commenti che associa ogni utente che ha commentato con tutti i commenti da lui postati, insieme a due mappe Stringa -> Long che rappresentano i voti positivi e negativi, dove la stringa rappresenta chi ha espresso il voto e il long il momento in cui il voto è stato espresso.

- **Comment:**

Rappresenta un commento, al momento della creazione la variabile che rappresenta il momento in cui viene creato viene settata automaticamente.

Classi remote :

- **FollowerCallbackImpl:**

Implementa l'interfaccia FollowerCallback e il metodo per aggiungere o rimuovere un determinato utente dalla propria lista follower memorizzata lato client e notificarlo su schermo.

- **ServerFollowerImpl:**

Implementa l'interfaccia ServerFollower, e i metodi di register e unregister dalla callback oltre al metodo che effettua le callback.

- **RegisterManagerImpl:**

Implementa l'interfaccia RegisterManager oltre al metodo per registrarsi al social network da remoto, che aggiunge una nuova entry alla mappa degli utenti utilizzando i dati di input.

Istruzioni d'uso:

Da dentro la cartella src compilare tutti i file presenti. Per usare il social network è necessario avviare prima il server altrimenti verrà visualizzato un messaggio che comunica che Winsone è correntemente offline.

Dal lato del server l'unico comando inseribile è quello di chiusura del server.

Dal lato client si potranno usare tutti i comandi previsti per il progetto, secondo una dinamica che prevede prima l'inserimento del comando e solo dopo, su richiesta da linea di comando, dei rispettivi parametri qualora ce ne fossero.

Comando per compilare :

```
javac -cp .;..\lib\gson-2.8.2.jar *.java
```

Comando per eseguire il client :

```
java -Djava.net.preferIPv4Stack=true -cp .;..\lib\gson-2.8.2.jar ClientMain ..\config\ConfigClient.txt
```

Comando per eseguire il server:

```
java -cp .;..\lib\gson-2.8.2.jar ServerMain ..\config\ConfigServer.txt
```